

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра инструментального и прикладного программного обеспечения (ИППО)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Архитектура, проектирование и разработка программных средств»

Практическая работа № 2

Атрибутный метод проектирования архитектуры

Студенты группы ИКМО-03-25 Фомичев Р.А. _____ (подпись)

Старший преподаватель *Синицын А.В.* _____ (подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

Проектирование архитектуры ИС «Учёт и обслуживание компьютерной техники» методом ADD

1. Цель работы и выбор системы

Цель работы — ознакомиться с методом атрибутного проектирования архитектуры (Attribute-Driven Design, ADD) и применить его для построения архитектуры программной системы, учитывающей требования к атрибутам качества. ADD строит архитектуру рекурсивной декомпозицией: на каждой итерации выбирается элемент системы, для него определяются архитектурные драйверы, подбираются тактики и паттерны, создаются дочерние элементы с обязанностями и интерфейсами, после чего проверяется соответствие требованиям.

Объект проектирования взят из практической работы № 1 – информационная система «Учёт и обслуживание компьютерной техники» (далее – система). ИТ-отдел из шести человек ведёт в ней реестр около 2500 единиц техники, принимает и выполняет заявки на обслуживание, планирует профилактику, проводит инвентаризацию и готовит акты для бухгалтерии. В плане первой работы проектирование архитектуры — задача **T02**; настоящая работа выполняет первую итерацию ADD для этой задачи. Требования ниже ссылаются на идентификаторы **F01**–**F14** и пункты 3.1–3.6 документа требований.

2. Определение требований

2.1. Функциональные требования

Из 14 функциональных требований первой работы выбраны три, которые сильнее других влияют на структуру системы: они затрагивают разные модули, внешние системы и фоновую обработку. Приоритет поставлен заказчиком (руководителем ИТ-отдела) по шкале «высокий / средний / низкий».

Таблица 1 – Функциональные требования, принятые на вход ADD

ID	Требование	Связь с ПР1	Приоритет
ФТ1	Сотрудник подаёт заявку на обслуживание своей техники; система присваивает номер З-ГГГГ-NNNNN, отправляет письмо с номером, ведёт заявку по статусам и контролирует сроки реакции и решения в рабочем времени.	F05, F06, F07	высокий
ФТ2	Специалист ИТ-отдела регистрирует единицу техники и оформляет её перемещение или смену ответственного; запись попадает в неизменяемую историю, формируется акт перемещения в PDF.	F01, F03	высокий

ID	Требование	Связь с ПР1	Приоритет
ФТ3	Специалист проводит инвентаризацию подразделения или кабинета, отмечая технику сканером штрихкода; система показывает расхождения «не на месте» и «не найдено».	F10	средний

2.2. Атрибуты качества

Требования к атрибутам качества записаны в форме сценариев «стимул – реакция» с шестью частями, как требует первый шаг ADD: источник стимула, стимул, артефакт, окружение, ответ и мера ответа (таблица 2). Значения мер взяты из разделов 2.2 и 3 документа требований.

Таблица 2 – Сценарии атрибутов качества

Часть сценария	АК1. Производительность	АК2. Безопасность	АК3. Доступность
Источник стимула	50 одновременно работающих пользователей	Пользователь с ролью «Сотрудник» (или чужой человек с его сеансом)	Отказ почтового сервера организации
Стимул	Поиск техники по нескольким условиям (F04), открытие карточки	Прямой запрос к API на изменение карточки техники или чтение чужой заявки, в обход интерфейса	Нужно отправить письмо о новой заявке, а почтовый сервер недоступен
Артефакт	Сервер приложений и база данных	API и подсистема контроля доступа	Модуль заявок и подсистема уведомлений
Окружение	Рабочее время; в базе 5000 единиц техники и 75 000 заявок	Штатная работа системы	Рабочее время, остальные компоненты исправны
Ответ	Результат возвращается постранично, карточка открывается	Запрос отклоняется, данные не меняются, попытка записывается в журнал аудита	Заявка регистрируется как обычно; письмо ставится в очередь и уходит после восстановления сервера
Мера ответа	Не менее 95 из 100 поисков – не более 2 с; карточка – не более 1 с (п. 3.2)	100 % запросов вне роли отклонены с кодом 403 по матрице ролей; запись в журнале есть для каждой попытки (F13, п. 3.3)	Номер заявки выдан не дольше 1 с; ни одно письмо не потеряно; письмо отправлено не позднее 5 мин после восстановления сервера

2.3. Ограничения

Ограничения – проектные решения, заданные извне; они сужают выбор на шаге 4 (таблица 3).

Таблица 3 – Ограничения проекта

ID	Ограничение	Источник в ПР1
О1	Один сервер организации: 4 ядра, 8 ГБ ОЗУ, 200 ГБ SSD, Linux; одна организация.	п. 3.4, 3.6
О2	СУБД PostgreSQL; клиент-серверное веб-приложение с REST API; работа в двух последних версиях Chrome, Firefox и Яндекс Браузера, форма заявки — на телефоне от 360 пикселей.	п. 1.4, 3.1

ID	Ограничение	Источник в ПР1
О3	Вход только через службу каталогов Active Directory по LDAPS, пароли в системе не хранятся; HTTPS с TLS не ниже 1.2; сеанс завершается через 30 минут бездействия.	п. 3.3
О4	Обмен с бухгалтерией — только файлами CSV, без подключения к её базе.	п. 1.4, F14
О5	Доступность не ниже 99 % в рабочее время (8:00–20:00 рабочих дней), потеря данных не более 24 ч, восстановление не более 4 ч; резервные копии шифруются.	п. 3.3, 3.4
О6	Один серверный разработчик, бюджет не более 3,5 млн ₽, передача системы до 01.03.2027; персональные данные — по 152-ФЗ.	п. 3.5, 3.6

3. Итерация ADD

3.1. Шаг 1. Подтверждение достаточности требований

Функциональные требования имеют критерии приёмки (таблица 1 первой работы) и приоритеты заказчика. Каждый сценарий атрибута качества содержит все шесть частей и измеримую меру. Ограничения согласованы с заказчиком в первой работе. Открытые вопросы первой работы (п. 4.1) — формы актов, формат файла обмена, модели сканера и принтера — меняют содержимое модулей, но не их состав и связи, поэтому на первую итерацию не влияют. Для сценария **АК3** не хватало значения «через сколько после восстановления уйдёт письмо»; мера в 5 минут выведена из требования **F05** («письмо не позднее 2 минут») с запасом на время отказа и подтверждена заказчиком. Требований достаточно, чтобы продолжать.

3.2. Шаг 2. Выбор элемента для декомпозиции

Система проектируется с нуля, поэтому на первой итерации единственный элемент для декомпозиции – вся система целиком, ей предъявляются все требования таблиц 1-3. Результатом итерации должны стать первые уровни декомпозиции: процессы и модули, их обязанности, интерфейсы и размещение на сервере.

3.3. Шаг 3. Архитектурные драйверы

Для каждого требования поставлена пара оценок: первая – важность для заинтересованных сторон (по приоритету заказчика), вторая – ожидаемо влияние на архитектуру (оценка архитектора). Высокая пара (Н, Н) означает требование, которое и важно, и определяет структуру. В драйверы выбраны шесть кандидатов с наибольшими парами (таблица 4).

Таблица 4 — Кандидаты в архитектурные драйверы

№	Кандидат	Источник	Пара оценок	Решение
Д1	Время поиска и открытия карточки при 50 пользователях	АК1	(H, H)	драйвер
Д2	Серверная проверка прав, вход через AD, журнал аудита	АК2, О3	(H, H)	драйвер
Д3	Устойчивость к отказу почты и сбоям сервера, восстановление	АК3, О5	(H, M)	драйвер
Д4	Заявки: статусы, сроки в рабочем времени, уведомления	ФТ1	(H, M)	драйвер
Д5	Реестр и неизменяемая история перемещений	ФТ2	(H, M)	драйвер
Д6	Один сервер, один серверный разработчик, срок	О1, О6	(H, H)	драйвер
—	Инвентаризация по штрихкоду	ФТ3	(M, L)	учтён на шаге 5
—	PostgreSQL и поддержка браузеров	О2	(H, L)	учтён как ограничение
—	Обмен CSV с бухгалтерией	О4	(M, L)	учтён на шаге 5

ФТ3 не попал в драйверы: сканер работает как клавиатура, поэтому отметка единицы – обычный запрос к API, и требование не меняет структуру системы. Ограничение **О2** предопределяет выбор СУБД и вида клиента, альтернатив для него нет.

3.4. Шаг 4. Выбор тактик и паттернов

Шаг выполнен по шести подшагам метода: определены проблемы проектирования для драйверов, для каждой составлен список альтернатив с различающими параметрами, выбрана подходящая альтернатива, установлена связь решений с драйверами, построены предварительные представления (раздел 4) и проведена оценка с устранением несоответствий. Проблемы, альтернативы и выбор сведены в таблицу 5.

Таблица 5 – Проблемы проектирования, альтернативы и выбор

Проблема	Драйверы	Альтернативы и различающий параметр	Выбор
Структура системы	Д6, Д5	а) монолит без модулей; б) модульный монолит; в) 4-6 микросервисов. Параметр: число развёртываемых единиц, расход памяти, нагрузка на одного разработчика	б) модульный монолит со слоями API – модули – доступ к данным
Вид клиента	Д1, О2	а) страницы формируются на сервере; б) одностраничное приложение (SPA) поверх REST API. Параметр: объём передаваемых данных, повторное использование API	б) SPA поверх REST API
Скорость чтения	Д1	а) индексы PostgreSQL, триграммный поиск, пагинация; б) отдельный поисковый движок; в) внешний кэш. Параметр: время запроса, расход памяти, число компонентов	а) индексы, pg_trgm, пагинация, кэш справочников в памяти процесса
Длительные и отложенные операции	Д3, Д4, Д6	а) выполнять в запросе пользователя; б) фоновый обработчик с очередью в PostgreSQL; в) внешний брокер сообщений. Параметр: время ответа, число компонентов	б) фоновый обработчик, очередь в таблице PostgreSQL

Проблема	Драйверы	Альтернативы и различающий параметр	Выбор
Контроль доступа и аудит	Д2	а) проверки в каждом обработчике; б) единая политика в слое прикладных сервисов; в) построчная защита (RLS) в СУБД. Аудит: триггеры БД или запись в той же транзакции. Параметр: полнота проверки, сложность	б) единая политика + фильтр «свои записи»; аудит в той же транзакции
Отказ внешних сервисов	Д3	а) синхронная отправка письма; б) транзакционный outbox с повтором и нарастающей паузой. Параметр: зависимость регистрации заявки от почты, риск потери письма	б) outbox с повтором
Обнаружение отказов и восстановление	Д3, Д6	а) ручной контроль; б) проверка /health, сигнал «жив», автоматический перезапуск; в) горячий резерв на втором сервере. Параметр: время простоя, стоимость	б) health check + перезапуск, восстановление из копии по сценарию
История изменений	Д5	а) правка записи на месте с журналом; б) текущее состояние + таблица событий только на добавление; в) полный event sourcing. Параметр: неизменяемость истории, сложность	б) состояние + неизменяемая таблица перемещений

Обоснование выбора. Микросервисы отклонены из-за ограничения **Д6**: на сервере с 8 ГБ ОЗУ каждая служба требует своего процесса, а один серверный разработчик не успеет сопровождать несколько развёртываний и межсервисные контракты. Монолит без модулей проще, но смешивает код четырёх модулей и мешает итерационной разработке из первой работы, где модули сдаются по очереди. Модульный монолит даёт одну развёртываемую единицу и при этом чёткие границы модулей с внутренними интерфейсами.

Отдельный поисковый движок и внешний кэш отклонены: объём данных мал (5000 единиц, 15 000 заявок в год), и индексы PostgreSQL обеспечивают время запроса в десятки миллисекунд, а дополнительные компоненты заняли бы 1–2 ГБ памяти. Внешний брокер сообщений тоже отклонён: очередь на таблице PostgreSQL с выборкой SELECT … FOR UPDATE SKIP LOCKED **[5]** выдерживает нужные десятки задач в минуту и не добавляет компонент, который надо резервировать отдельно.

Построчная защита в СУБД отклонена, потому что роли берутся из AD, а приложение работает через общий пул соединений: перенос роли пользователя в сеанс СУБД усложняет код без выигрыша. Единая политика доступа вызывается слоем прикладных сервисов перед каждой операцией, поэтому обход интерфейса прямым запросом к API ничего не даёт. Журнал аудита пишется в той же транзакции, что и изменение: если запись аудита не удалась, не сохраняется и изменение. Триггеры БД отклонены, так как не знают автора действия.

Синхронная отправка письма отклонена: при недоступном почтовом сервере регистрация заявки ждала бы тайм-аута или завершалась ошибкой. В паттерне

«транзакционный outbox» письмо записывается в таблицу outbox в той же транзакции, что и заявка, а фоновый обработчик отправляет его отдельно и повторяет при ошибке. Горячий резерв на втором сервере отклонён ограничением **О1** и бюджетом; вместо него выбраны тактики обнаружения отказа (проверка /health, сигнал «жив» от фонового обработчика) и восстановления (автоматический перезапуск служб, восстановление из резервной копии по готовому сценарию).

Оценка решения. Чтобы проверить выбор до реализации, составлены две модели: распределение ресурсов сервера (таблица 6) и бюджет времени поискового запроса (таблица 7). Значения – оценки по опыту, их подтверждает нагрузочное испытание задачи **Т16** первой работы.

Таблица 6 – Распределение ресурсов сервера (оценка)

Потребитель	ОЗУ, ГБ	Диск, ГБ
Операционная система и служебные программы	1,0	15
PostgreSQL: общий буфер	2,0	—
PostgreSQL: рабочая память 20 соединений	0,7	—
PostgreSQL: данные за 5 лет / журнал WAL и временные файлы	—	20 / 5
Сервер приложений, 4 процесса по 0,3 ГБ	1,2	—
Фоновый обработчик	0,4	—
Nginx	0,1	—
Пик формирования отчётов XLSX и PDF	0,5	—
Журналы работы и локальная копия последней резервной копии	—	5 + 20
Итого / доступно / запас	5,9 / 8,0 / 2,1	65 / 200 / 135

Таблица 7 – Бюджет времени поиска техники, 95-й процентиль (оценка)

Этап	Время, мс
Сеть и установление TLS	100
Nginx: приём и передача запроса	5
Проверка сеанса и прав	20
SQL-запрос по индексам: 50 строк страницы и подсчёт найденных	150
Формирование JSON	30
Отрисовка таблицы в браузере	300
Итого / требование АК1 / запас	605 / 2000 / 1395

Нагрузка: 50 пользователей при одном действии в 10 секунд дают в среднем 5 запросов в секунду, в пике — до 20. При 50 мс процессорного времени на запрос это 1 ядро из 4, то есть около 25 % мощности сервера. Память и диск укладываются с запасом 26 % и 67 %. Обе модели показывают, что модульный монолит на одном сервере удовлетворяет драйверам **Д1** и **Д6**.

Устранение несоответствия. Проверка драйвера **Д3** выявила противоречие внутри требований первой работы. Доступность 99 % за месяц при 12-часовом рабочем дне и примерно 21 рабочем дне означает не более 2,5 ч простоя в месяц (1 % от 252 ч), а допустимое время восстановления — 4 ч. Значит, один полный отказ сервера, восстановленный «в норматив», уже нарушит доступность. Для программных сбоев противоречия нет: проверка /health каждые 30 с и перезапуск после двух неудачных проверок дают простой около 1,5 мин. Для отказа сервера рассмотрены три варианта: второй сервер в горячем резерве (отклонён ограничениями **О1** и **О6**); заранее подготовленный образ виртуальной машины и сценарий восстановления, сокращающие восстановление до 1 ч 25 мин (развёртывание из образа — 20 мин, установка приложения сценарием — 10 мин, восстановление базы до 20 ГБ из копии — 40 мин, проверка — 15 мин); согласование с заказчиком, что полная потеря сервера считается аварией и не входит в расчёт доступности. Выбран второй вариант. Третий передан заказчику как открытый вопрос, так как время восстановления зависит от того, есть ли у организации запасные мощности виртуализации.

3.5. Шаг 5. Архитектурные элементы и обязанности

По выбранным паттернам созданы экземпляры элементов (таблица 8). Обязанности выведены из типа элемента и из функциональных требований родительского элемента — системы.

Таблица 8 – Элементы архитектуры и их обязанности

№	Элемент	Обязанности	Драйверы, требования
Э1	Веб-клиент (SPA)	Экраны реестра, заявок, ЗИП, ТО, инвентаризации и отчётов; адаптивная форма заявки для телефона; на экране инвентаризации фокус в поле штрихкода; показ ошибок ввода рядом с полем.	Д1, ФТ1–ФТ3, О2
Э2	Обратный прокси (Nginx)	Завершение HTTPS (TLS ≥ 1.2), раздача файлов клиента, передача запросов к API, ограничение размера загружаемых файлов.	Д2, О3
Э3.1	API-слой	Приём REST-запросов, проверка входных данных, пагинация, единый формат ошибок, вызов подсистемы безопасности перед каждой операцией.	Д1, Д2
Э3.2	Подсистема безопасности	Вход через AD, сеанс с завершением через 30 мин бездействия, проверка роли и владельца записи, запись журнала аудита в транзакции вызывающего модуля.	Д2, F13
Э3.3	Модуль «Учёт техники»	Карточка и комплект рабочего места, уникальность инвентарного номера, перемещения и смена ответственного с записью в неизменяемую историю, акт перемещения.	Д5, ФТ2

№	Элемент	Обязанности	Драйверы, требования
Э3.4	Модуль «Заявки и обслуживание»	Нумерация заявок, статусная модель с допустимыми переходами, расчёт сроков в рабочем календаре, журнал работ, учёт ЗИП, график ТО, постановка писем в outbox.	Д3, Д4, ФТ1
Э3.5	Модуль «Инвентаризация и списание»	Ведомость по подразделению или кабинету, приём отметок штрихкода, расчёт расхождений, процесс списания с утверждением руководителем.	ФТ3, F11
Э3.6	Модуль «Отчёты и обмен»	Отчёты XLSX и PDF, проверка и загрузка CSV-справочника основных средств, выгрузка актов для бухгалтерии.	F12, F14, О4
Э4	Фоновый обработчик	Отправка писем из outbox с повтором, контроль просрочки и создание заявок ТО, суточная синхронизация сотрудников из AD, импорт CSV, сигнал «жив» раз в 30 с.	Д3, Д4, Д6
Э5	СУБД PostgreSQL	Хранение данных учёта, истории, журнала аудита, таблиц outbox и очереди задач; индексы для поиска; ограничения целостности (уникальность номера, неотрицательный остаток ЗИП).	Д1, Д5, О2
Э6	Адаптеры интеграции	LDAP-клиент (проверка входа, список сотрудников), SMTP-клиент, работа с каталогом обмена CSV; тайм-ауты и перевод ошибок внешних систем во внутренние.	Д3, О3, О4
Э7	Мониторинг и резервное копирование	Проверка /health каждые 30 с, перезапуск служб, ночная зашифрованная копия базы с хранением 30 дней, сценарий восстановления, ежеквартальная проверка восстановления.	Д3, О5

В конце шага каждое функциональное требование родительского элемента должно выражаться последовательностью обязанностей дочерних элементов (таблица 9).

Таблица 9 – Реализация функциональных требований элементами

ФТ	Последовательность обязанностей
ФТ1	Э1 – форма заявки → Э2 – HTTPS → Э3.1 – проверка данных → Э3.2 – роль «Сотрудник», техника принадлежит заявителю → Э3.4 – номер, сроки по приоритету, статус «Новая» → Э5 – в одной транзакции заявка, запись аудита и письмо в outbox → Э3.1 – номер заявки клиенту → Э4 – выборка письма из outbox → Э6 – отправка через SMTP; далее Э4 раз в минуту проверяет сроки и при просрочке ставит письмо руководителю.
ФТ2	Э1 – карточка или форма перемещения → Э3.1 → Э3.2 – роль «Техник» → Э3.3 – проверка уникальности номера, новое место и ответственный, запись в таблицу истории (только добавление) → Э5 – транзакция с записью аудита → Э3.6 – акт перемещения в PDF с инвентарными номерами.
ФТ3	Э1 – экран ведомости, сканер вводит код в поле → Э3.1 → Э3.2 – роль «Техник» → Э3.5 – поиск единицы по коду и сравнение кабинета: «на месте», «не на месте» или неизвестный код → Э5 – отметка в ведомости → Э3.5 – при закрытии ведомости список «не найдено» и выгрузка расхождений.

3.6. Шаг 6. Интерфейсы элементов

Интерфейс в ADD описывает не только сигнатуры операций, но и то, что элемент предоставляет и что требует от других: предусловия, постусловия, передаваемые данные и требования к качеству. Основные интерфейсы приведены в таблице 10; полный перечень методов API фиксируется при детальной декомпозиции модулей.

Таблица 10 – Интерфейсы элементов

Элемент	Операции	Предоставляет / требует	Качество
Э3.1 API	GET /api/v1/assets?inv=&type=&dept=&owner=&status=&page=; GET, POST, PATCH /api/v1/assets/{id}; POST /api/v1/assets/{id}/moves; POST /api/v1/requests; PATCH /api/v1/requests/{id}/status; POST /api/v1/inventories/{id}/scans	JSON по HTTPS; ошибки: 400 (поле и причина), 401, 403, 409 (повтор номера, недопустимый переход). Требует действующий сеанс	Серверная часть ≤ 1 с для 95 % запросов
Э3.2 Безопасность	authenticate(логин, пароль) → сеанс; authorize(пользователь, действие, объект) → разрешено / запрещено; audit(автор, действие, объект, старое, новое)	Пароль не сохраняется; audit выполняется только внутри транзакции вызывающего модуля. Требует Э6 (LDAP)	authorize ≤ 20 мс
Э3.3 Учёт техники	registerAsset(данные); moveAsset(id, куда, ответственный) → акт; getHistory(id)	Постусловие moveAsset: новая запись истории, старые не меняются	—
Э3.4 Заявки	createRequest(техника, описание, приоритет) → номер; changeStatus(id, статус); computeDeadlines(приоритет, время создания)	Предусловие changeStatus: переход допустим статусной моделью; сроки считаются только в рабочем времени	createRequest ≤ 1 с
Э3.5 Инвентаризация	startInventory(подразделение | кабинет); registerScan(ведомость, код) → результат; closeInventory(ведомость) → расхождения	Повторная отметка той же единицы не создаёт дубля	registerScan ≤ 0,5 с
Э4 Обработчик	enqueue(тип, данные) — запись задачи или письма в той же транзакции; сигнал «жив» в таблицу состояния	Опрос очереди каждые 15 с; повтор письма через 30 с, 1, 2, 4 мин, далее раз в 5 мин в течение 3 суток	Письмо ≤ 2 мин при исправной почте
Э6 Адаптеры	ldapBind(логин, пароль); ldapListUsers(); smtpSend(письмо); readCsv(файл), writeCsv(акты)	LDAPS, порт 636, тайм-аут 5 с; SMTP, тайм-аут 10 с; формат CSV — по согласованию с бухгалтерией	Ошибка внешней системы не роняет процесс
Э7 Мониторинг	GET /health → 200 или 503 с состоянием БД и временем последнего сигнала «жив»	Перезапуск службы после двух неудачных проверок подряд	Обнаружение ≤ 60 с

3.7. Шаг 7. Проверка соответствия требованиям и ограничениям

Проверка показывает, что каждый драйвер закрыт решениями шага 4 и обязанностями элементов шага 5, и указывает, чем это будет подтверждено (таблица 11).

Таблица 11 – Проверка архитектурных драйверов

№	Как удовлетворён	Элементы	Подтверждение
Д1	Индексы и триграммный поиск, пагинация, кэш справочников; бюджет 605 мс при норме 2000 мс	Э1, Э3.1, Э5	Нагрузочный тест Т16 на 50 пользователях
Д2	Единая политика доступа на сервере, вход через AD по LDAPS, аудит в той же транзакции, HTTPS	Э2, Э3.2, Э6	Тест матрицы ролей: все запросы вне роли — 403
Д3	Outbox с повтором; /health, сигнал «жив», перезапуск; восстановление по сценарию за 1 ч 25 мин	Э3.4, Э4, Э6, Э7	Тест с отключённой почтой; квартальная проверка восстановления

№	Как удовлетворён	Элементы	Подтверждение
Д4	Статусная модель и расчёт сроков в модуле заявок; проверка просрочки фоновым обработчиком	Э3.4, Э4	Сценарии F05–F07 в тестах T12
Д5	Таблица истории только на добавление, уникальный индекс инвентарного номера	Э3.3, Э5	Сценарии F01, F03 в тестах T08
Д6	Одна развёртываемая единица и один язык серверной части; ресурсы — 5,9 ГБ из 8 ГБ	все	Таблица 6, замер на стенде T05

Ограничения соблюдены: используются PostgreSQL и браузерный клиент (**О2**); пароли не хранятся, вход только через AD (**О3**); с бухгалтерией система обменивается только файлами (**О4**); все компоненты размещаются на одном сервере (**О1**) и не требуют лицензий (**О6**). Остаточный риск: при недоступности AD новые пользователи не смогут войти, работают только открытые сеансы. Кэширование паролей устранило бы риск, но нарушает **О3**, поэтому риск принят и сообщён заказчику.

Требования уточнены и превращены в ограничения для дочерних элементов: серверная часть API отвечает на поиск не дольше 1 с, а SQL-запрос — не дольше 0,3 с для 95 % запросов; фоновый обработчик опрашивает очередь не реже чем раз в 15 с и повторяет отправку не реже чем раз в 5 мин; мониторинг обнаруживает сбой не дольше чем за 60 с; восстановление из копии выполняется сценарием не дольше 1,5 ч; каждый модуль меняет данные только через свои прикладные сервисы, вызывающие authorize и audit.

3.8. Шаг 8. Фиксация результата и элементы для дальнейшей декомпозиции

Первая итерация разложила систему на семь элементов верхнего уровня и шесть частей сервера приложений, каждой назначены обязанности, интерфейсы, требования к качеству и ограничения. Следующие итерации возвращаются к шагу 2 и выбирают очередной элемент по риску и сложности (таблица 12).

Таблица 12 – Элементы после первой итерации и план декомпозиции

№	Элемент	Что предстоит решить	Итерация
Э3.4	Заявки и обслуживание	Статусная модель, рабочий календарь и праздники, расчёт сроков при паузе «Ожидает запчасть», связь заявок, работ и ЗИП	2 — наибольший риск
Э3.2	Подсистема безопасности	Матрица «роль — действие — объект», хранение сеанса, состав журнала аудита, меры защиты персональных данных по уровню защищённости	3
Э4	Фоновый обработчик	Типы задач, порядок повторов, защита от двойного выполнения, сигнал «жив»	3
Э3.5, Э3.6	Инвентаризация, отчёты и обмен	После ответа заказчика о формах актов и формате CSV	4

№	Элемент	Что предстоит решить	Итерация
Э1, Э2, Э3.1, Э3.3, Э5, Э6, Э7	Остальные элементы	Решаются на уровне детального проектирования без отдельной итерации ADD	—

4. Визуализация архитектуры

На рисунке 1 показана архитектура системы после первой итерации: элементы, их обязанности и связи. Все элементы, кроме веб-клиента, размещаются на одном сервере организации; внешние системы показаны справа, подписи у стрелок называют протокол.

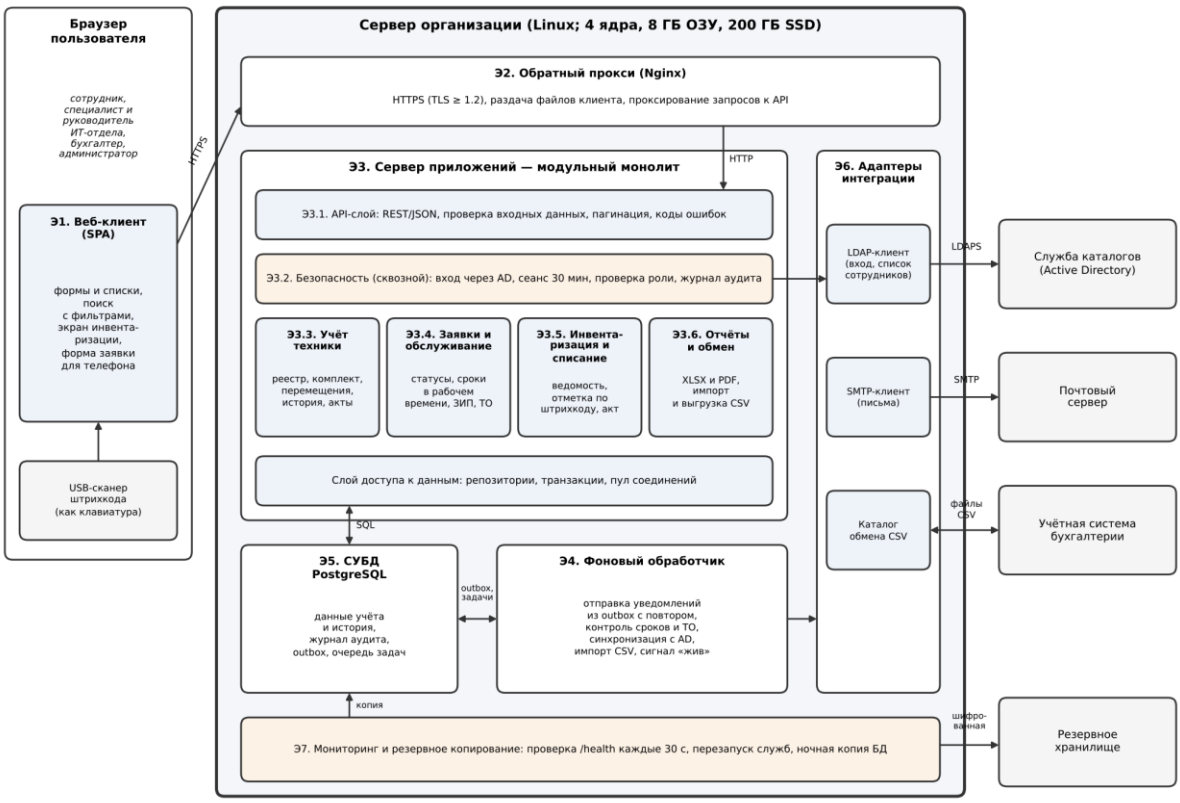


Рисунок 1 – Архитектура ИС учёта и обслуживания компьютерной техники

Пользователь работает в браузере с веб-клиентом **Э1**, который обращается к серверу только по HTTPS через прокси **Э2**. Сервер приложений **Э3** устроен слоями: API-слой принимает запросы, подсистема безопасности проверяет права для каждой операции, четыре прикладных модуля соответствуют модулям из первой работы, слой доступа к данным работает с PostgreSQL. Фоновый обработчик **Э4** берёт из базы письма и задачи и обращается к внешним системам через адаптеры **Э6**. Мониторинг **Э7** следит за службами и делает зашифрованные копии базы в резервное хранилище.

На рисунке 2 показано, как решения шагов 4–6 выполняют сценарий **АК3**: регистрация заявки не зависит от почтового сервера, а письмо доставляется после его восстановления.

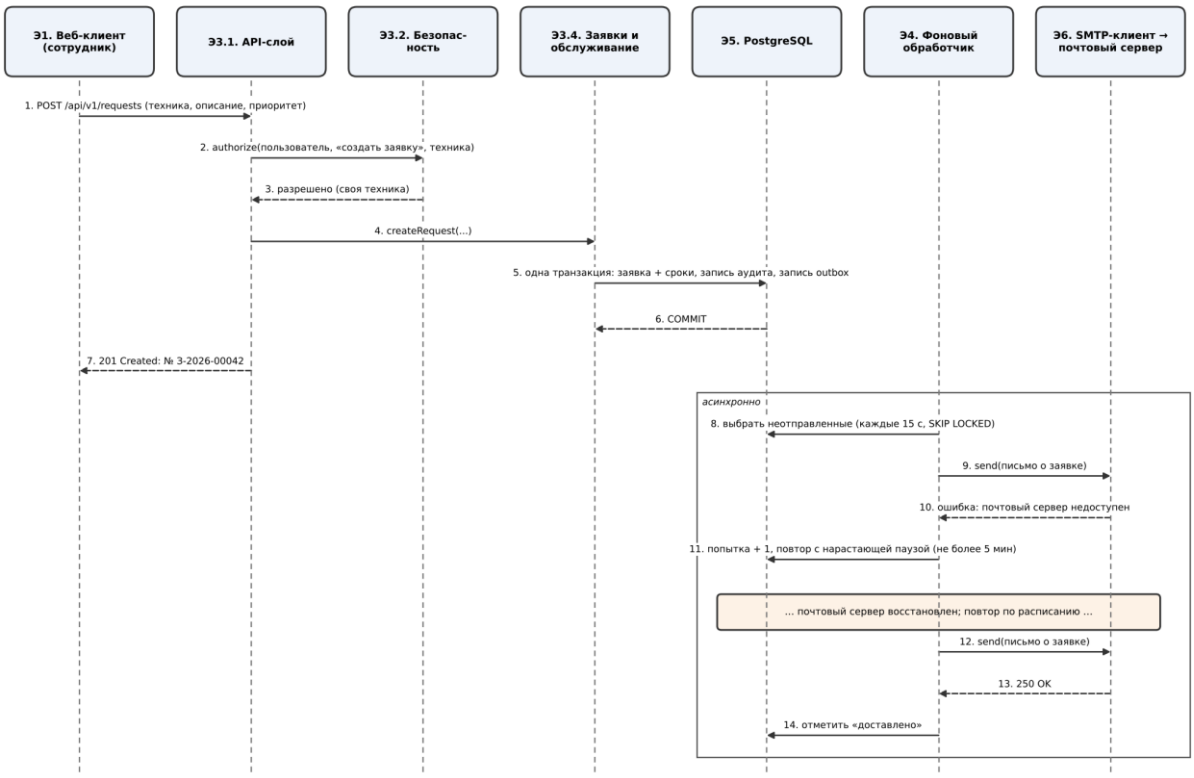


Рисунок 2 – Последовательность подачи заявки при отказе почтового сервера

Сотрудник получает номер заявки сразу после фиксации транзакции (сообщения 1–7). Письмо записано в outbox в той же транзакции, поэтому не теряется, даже если сервер перезапустится. Фоновый обработчик отправляет письмо асинхронно, при ошибке увеличивает счётчик попыток и повторяет отправку с нарастающей паузой не более 5 минут, после чего отмечает письмо доставленным (сообщения 8–14).

5. Ссылки

[1] Wood W. G. A Practical Example of Applying Attribute-Driven Design (ADD), Version 2.0. CMU/SEI-2007-TR-005. Pittsburgh: Software Engineering Institute, 2007.

[2] Wojcik R., Bachmann F., Bass L. et al. Attribute-Driven Design (ADD), Version 2.0. CMU/SEI-2006-TR-023. Pittsburgh: Software Engineering Institute, 2006.

[3] Bass L., Clements P., Kazman R. Software Architecture in Practice. 2nd ed. Boston: Addison-Wesley, 2003.

[4] PostgreSQL Documentation. SELECT: The Locking Clause: https://www.postgresql.org/docs/current/sql-select.html#SQL-FOR-UPDATE-SHARE

[5] Richardson C. Pattern: Transactional outbox: https://microservices.io/patterns/data/transactional-outbox.html

Заключение

Для системы учёта и обслуживания компьютерной техники выполнена первая итерация метода ADD. На вход приняты три функциональных требования, три сценария атрибутов качества и шесть ограничений; из них выбраны шесть архитектурных драйверов. Выбраны модульный монолит с SPA-клиентом, поиск на индексах PostgreSQL, фоновый обработчик с очередью в базе, единая серверная политика доступа с аудитом в той же транзакции, транзакционный outbox для уведомлений, а также обнаружение отказов с перезапуском и восстановлением по сценарию. Оценка показала запас по времени поиска (605 мс при норме 2000 мс) и по ресурсам сервера (5,9 ГБ из 8 ГБ) и выявила противоречие между доступностью 99 % и временем восстановления 4 ч; оно устранено сокращением восстановления до 1 ч 25 мин и передано заказчику как открытый вопрос. Следующая итерация декомпозирует модуль «Заявки и обслуживание».